

БГУИР
Кафедра ЭВМ

Отчет по лабораторной работе №3
Тема: «Работа со строками»

Проверила:
Герман Ю.О.

Выполнили:
Шарай П.Ю.
Соломко В.Ю.

Минск 2023

Цель

Изучить технику работы со строками в Scala.

1. Краткие теоретические сведения

Scala предоставляет богатый набор функций для работы со строками. Вот некоторые расширенные функции, которые можно использовать со строками в Scala:

1. `replaceAll` – эта функция используется для замены всех вхождений строки другой строкой. Функция принимает два аргумента: первый аргумент — это заменяемое регулярное выражение, а второй аргумент — строка замены.

2. `split` – эта функция используется для разделения строки на массив подстрок на основе разделителя. Функция принимает один аргумент — строку-разделитель.

3. `startsWith` и `endsWith` – эти функции используются для проверки того, начинается или заканчивается строка заданной подстрокой. Функции принимают один аргумент — проверяемую подстроку.

4. `substring` – выделяет подстроку из строки.

5. `toArray` – преобразует строку в массив символов.

6. `toLowerCase` и `toUpperCase` – преобразует символы строки в верхний и нижний регистр соответственно.

7. `trim` – отсекает концевые пробелы.

8. `indexOf` и `lastIndexOf` – получает первый и последний индекс подстроки в строке (то есть номер позиции, с которой начинается подстрока).

9. `charAt` – определяет символ, стоящий на указанной позиции.

10. `stripMargin` – удаляет ведущие пробелы перед строкой.

Регулярные выражения (Regex) – это строки, задающие шаблон для поиска определенных фрагментов в тексте. Помимо поиска, с помощью специальных Regex-шаблонов можно манипулировать текстовыми фрагментами – удалять и изменять подстроки частично или полностью.

Для вывода коллекции можно использовать `mkString(" , ")`.

Пример 1

Применение функции `replaceAll`:

```
val str = "Hello, World!"  
val newStr = str.replaceAll("o", "a")  
println(newStr) // "Hella, World!"
```

Пример 2

Применение функции `split`:

```
val str = "apple,banana,orange"  
val arr = str.split(",")
```

```
println(arr.mkString(" ")) // "apple banana orange"
```

Пример 3

Применение функций startsWith и endsWith:

```
val str = "Hello, World!"  
println(str.startsWith("Hello")) // true  
println(str.endsWith("!")) // true
```

Пример 4

Применение функции substring:

```
val str = "Hello, World!"  
val subStr = str.substring(7, 12)  
println(subStr) // "World"
```

Пример 5

Применение функции toArray:

```
val str = "Hello, World!"  
val arr = str.toCharArray()  
println(arr.mkString(" ")) // "H e l l o ,   W o r l d !"
```

Пример 6

Применение функций toLowerCase и toUpperCase:

```
val str = "Hello, World!"  
println(str.toLowerCase) // "hello, world!"  
println(str.toUpperCase) // "HELLO, WORLD!"
```

Пример 7

Применение функции trim:

```
val str = " Hello, World! "  
println(str.trim) // "Hello, World!"
```

Пример 8

Применение функций indexOf и lastIndexOf:

```
val str = "Hello, World!"  
println(str.indexOf("o")) // 4  
println(str.lastIndexOf("o")) // 8
```

Пример 9

Применение функции charAt:

```
val str = "Hello, World!"  
println(str.charAt(7)) // 'W'
```

Пример 10

Применение функции stripMargin:

```
val str =  
  """  
    |Hello,  
    |World!  
    |""".stripMargin  
println(str) // "Hello,\nWorld!\n"
```

Пример 11

В Scala регулярные выражения представлены классом `scala.util.matching.Regex`, который предоставляет множество методов для сопоставления строк и управления ими на основе регулярных выражений.

Вот пример, демонстрирующий некоторые основные функции регулярных выражений в Scala:

```
val regex = """(\d{3})-(\d{2})-(\d{4})""".r  
  
val str1 = "123-45-6789"  
val str2 = "abc-12-3456"  
  
val match1 = regex.findFirstMatchIn(str1)  
val match2 = regex.findFirstMatchIn(str2)  
  
match1 match {  
  case Some(m) => println(s"Match found: ${m.group(0)}")  
  case None => println("No match found")  
}  
  
match2 match {  
  case Some(m) => println(s"Match found: ${m.group(0)}")  
  case None => println("No match found")  
}
```

В этом примере мы определяем шаблон регулярного выражения, который соответствует номеру социального страхования в формате XXX-XX-XXXX, где X — цифра. Затем мы пытаемся сопоставить этот шаблон с двумя разными строками: «123-45-6789» и «abc-12-3456».

Метод `findFirstMatchIn` возвращает объект `Option[Match]`, представляющий первое совпадение шаблона в заданной строке, если таковое имеется. Мы используем сопоставление с образцом, чтобы извлечь совпадающую подстроку из объекта `Match` и распечатать ее.

Когда мы запускаем этот пример, мы получаем следующий вывод:

```
Match found: 123-45-6789  
No match found
```

В этом случае первая строка соответствует шаблону регулярного выражения, поэтому мы получаем объект соответствия с совпадающей подстрокой «123-45-6789». Вторая строка не соответствует шаблону, поэтому мы получаем объект None вместо объекта соответствия.

Обратите внимание, что регулярные выражения могут быть довольно мощными и сложными, и в классе Regex доступно гораздо больше функций и методов для работы с ними.

2. Ход работы

Задание для всех

Вывести суммарное число всех гласных в собственном тексте.

```
def vowelsCount(): Unit = {  
  print("Input text: ")  
  var text: String = scala.io.StdIn.readLine()  
  var regex: Regex = "[aeiouyAEIOUY]".r  
  var matches = regex.findAllMatchIn(text)  
  println("Vowels count: " + matches.size)  
}
```

Результат представлен на рисунке 4.0.

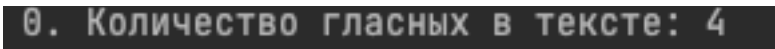


Рисунок 4.0 – Результат выполнения

Задание 4.1

Дан текст: ‘Hello to everybody’. С помощью техники регулярных выражений заменить латинские буквы на русские (или на цифры, если русский шрифт не поддерживается)

```
def latinToRussian(): Unit = {  
  var text: String = "Hello to everybody"  
  println("Source: " + text)  
  var regex: Regex = "[a-zA-Z]".r  
  var newText: String = regex.replaceAll(text, { c => (c.toString + 32).toString })  
  println("Result: " + newText)  
}
```

Результат представлен на рисунке 4.1.

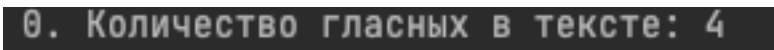


Рисунок 4.1 – Результат выполнения

Задание 4.2

Найти в тексте “When executing the exercise extract all extra words” все слова, начинающиеся на ext.

```
def getStartedWith(): Unit = {  
  var text: String = "When executing the exercise extract all extra words"  
  println("Source: " + text)  
  var regex: Regex = """"\bext\w+\b""".r  
  var matches = regex.findAllMatchIn(text)  
  println("Results:")  
  for(m <- matches) {  
    println(m.group(0))  
  }  
}
```

Результат представлен на рисунке 4.2.



2. Слова, начинающиеся на 'ext': extract, extra

Рисунок 4.2 – Результат выполнения

Задание 4.3

В тексте ‘A big round ball fall to the ground’ заменить артикль the на a.

```
def changeWord(): Unit = {  
  var text: String = "A big round ball fall to the ground"  
  println("Source: " + text)  
  var regex: Regex = """"\bthe\b""".r  
  var newText: String = regex.replaceAll(text, "a")  
  println("Result: " + newText)  
}
```

Результат представлен на рисунке 4.3.



3. Текст с замененными артиклями: A big round ball fall to a ground

Рисунок 4.3 – Результат выполнения

Задание 4.4

Записать все слова в тексте в обратном порядке.

```
def reverseText(): Unit = {  
  var text: String = "A big round ball fall to the ground"  
  println("Source: " + text)  
  var regex: Regex = """"\b\w+\b""".r  
  var words = regex.findAllMatchIn(text).map(_.group(0))  
}
```

```

var newText: String = ""

for (word <- words){
    newText = word + " " + newText
}
println("Result: " + newText)
}

```

Результат представлен на рисунке 4.4.

4. Текст с обратным порядком слов: ground the to fall ball round big A

Рисунок 4.4 – Результат выполнения

Задание 4.5

Дан текст: 'Hello to everybody'. Выбросить все гласные.

```

def removeVowels(): Unit = {
    var text: String = "Hello to everybody"
    println("Source: " + text)
    var regex: Regex = "[aeiouyAEIOUY]".r
    var newText = regex.replaceAll(text, "")
    println("Result: " + newText)
}

```

Результат представлен на рисунке 4.5.

5. Текст без гласных: Hll t vrybdy

Рисунок 4.5 – Результат выполнения

Задание 4.6

Дан текст: 'Hello to everybody'. Удвоить каждую букву в слове

```

def doubleSymbol(): Unit = {
    var text: String = "Hello to everybody"
    println("Source: " + text)
    var regex = "\\w".r
    var newText: String = regex.replaceAll(text, { m => m.group(0) + m.group(0) })
    println("Result: " + newText)
}

```

Результат представлен на рисунке 4.6.

6. Текст с удвоенными буквами: HHeellllloo ttoo eevvveerrgyybboddy

Рисунок 4.6 – Результат выполнения

Задание 4.7

Дан текст: 'Hello to everybody'. Удалить все вхождения буквы y

```
def removeSymbol(): Unit = {  
    var text: String = "Hello to everybody"  
    println("Source: " + text)  
    var regex: Regex = """"y"""".r  
    var newText: String = regex.replaceAll(text, "")  
    println("Result: " + newText)  
}
```

Результат представлен на рисунке 4.7.



Рисунок 4.7 – Результат выполнения

Задание 4.8

Дан текст: 'Hello to everybody'. Вставить слова with heartness чтобы получить Hello with heartness to everybody

```
def insertText(): Unit = {  
    var text: String = "Hello to everybody"  
    println("Source: " + text)  
    var regex: Regex = """"\bto\b"""".r  
    var newText: String = regex.replaceAll(text, "with heartness to")  
    println("Result: " + newText)  
}
```

Результат представлен на рисунке 4.8.

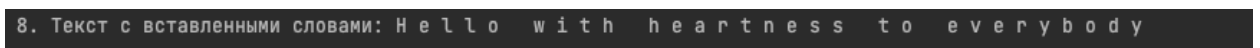


Рисунок 4.8 – Результат выполнения

3. Вывод

В ходе выполнения лабораторной работы нами были изучены техники работы со строками в Scala.